



سرفصل‌های مقاله

نصب آناکوندا

آرایه در NumPy

Broadcasting یا انتشار همگانی در Numpy

شکل آرایه (Shape of Array)

آرایه با مقادیر صفر و یا با مقادیر یک در NumPy

numpy.reshape و numpy.flatten در پایتون

اتصال آرایه‌ها به هم در NumPy (Stacking)

تقسیم آرایه در NumPy (Splitting)

پیمایش آرایه در NumPy

جبرخطی در NumPy

تولید اعداد تصادفی در NumPy

کار با تاریخ و زمان در NumPy

numpy.asarray در پایتون

numpy.arange در پایتون

numpy.linspace و numpy.logspace در پایتون

اندیس دهی (Indexing) و برش (Slicing) آرایه‌های NumPy در پایتون

توابع آماری NumPy

ضرب در پایتون به کمک NumPy

تفاوت میان View و Copy در NumPy

نتیجه گیری

◀ نمایش بیشتر

اگر به دنبال یادگیری علم داده توسط پایتون هستید، چاره ای جز یادگیری نحوه ی کار با کتابخانه‌های پایتون که به طور تخصصی برای این منظور طراحی شده اند، ندارید. NumPy، Matplotlib، Pandas، Seaborn، SiPy، Tensorflow و ... نمونه ای از این پکیج‌های نرم افزاری برای محاسبات ریاضی اند که شما در کار با داده‌ها و پردازش شان باید از آن‌ها استفاده کنید. در مجموعه مقالات **آموزش علم داده با پایتون** به تشریح کاربردی و مفید این پکیج‌ها و نحوه ی کار با آن‌ها خواهیم پرداخت. NumPy که مخفف Numerical Python است، برای محاسبات عددی متنوعی در پایتون به کار می‌رود. محاسبات به کمک آرایه‌ها در Numpy سرعت خوبی دارند و علاوه بر آن، توابع این پکیج در ساخت پکیج‌های محاسباتی دیگر مورد استفاده قرار گرفته است. محیط پیشنهادی ما برای کار با پکیج‌های محاسباتی در پایتون آناکوندا (Anaconda) است. از این رو نحوه ی نصب آن را نیز آموزش داده ایم. در مقاله ی آموزش کامل کتابخانه NumPY ، سعی بر آن شده تا مجموعه ای از متدها و توابع Numpy به طور کامل تشریح شود. تعداد توابع این نرم افزار بسیار گسترده است و شما در طول انجام پروژه‌های بزرگ به انواع مختلفی از توابع نیاز خواهید داشت که می‌توانید از طیف گسترده ای از پکیج‌ها استفاده کنید. یادگیری Numpy در عین سادگی مفاهیم و قدرتمندی بالای آن، برای کار با سایر کتابخانه‌های محاسباتی پایتون ضروری است. در ادامه به بررسی توابع این بسته ی نرم افزاری می‌پردازیم.

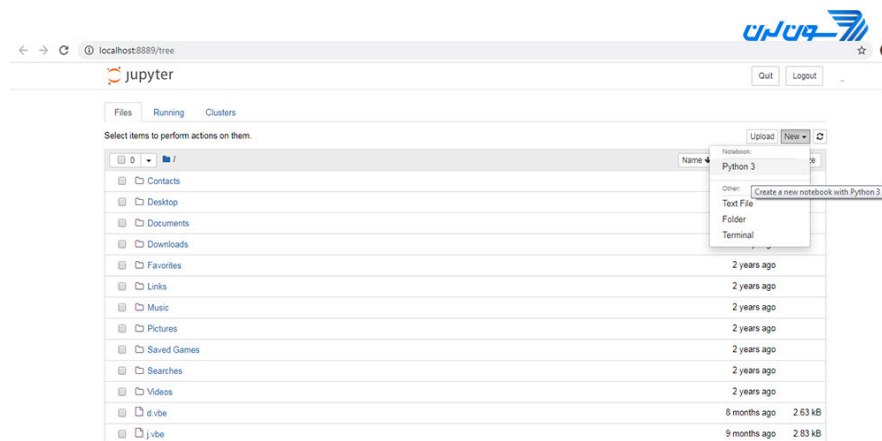
نصب آناکوندا

در این قسمت از مقاله ی آموزش کامل کتابخانه NumPY ، با آموزش نصب آناکوندا با ما همراه باشید. آناکوندا پکیجی است که شامل تعدادی از ماژول‌ها و کتابخانه‌های پایتون است که IDEهایی برای کدزنی در اختیار شما قرار می‌دهد و Jupyter Notebook یکی از آن‌هاست. با نصب آناکوندا بسیاری از کتابخانه‌های محاسباتی پایتون مثل NumPy، Pandas، Matplotlib و ... به طور خودکار نصب می‌شوند. برای دانلود و نصب آناکوندا متناسب با سیستم خود و همچنین نسخه ی پایتون نصب شده روی سیستم خودتان می‌توانید به [سایت اصلی آناکوندا](#) مراجعه کنید و اگر به هر دلیلی موفق به بازکردن سایت نشدید می‌توانید با جستجو در اینترنت، آناکوندای مناسب سیستم عامل خودتان را بیابید و نصب کنید. (محیطی که ما با آن کار می‌کنیم سیستم عامل ویندوز و پایتون ورژن ۳.۷ است.) Jupyter امکان کارکردن با پایتون را در خلال نوت بوک (Notebook) مجازی فراهم می‌کند و در دنیای علم داده بسیار مشهور است. ژوپیتتر این امکان را فراهم می‌کند که کدها، تصاویر، نمودارها را کنار هم و در یک صفحه ببینید و از این رو برای درک مسائل علم داده به خصوص برای دانشجویان بسیار کاربردی و مفید است.

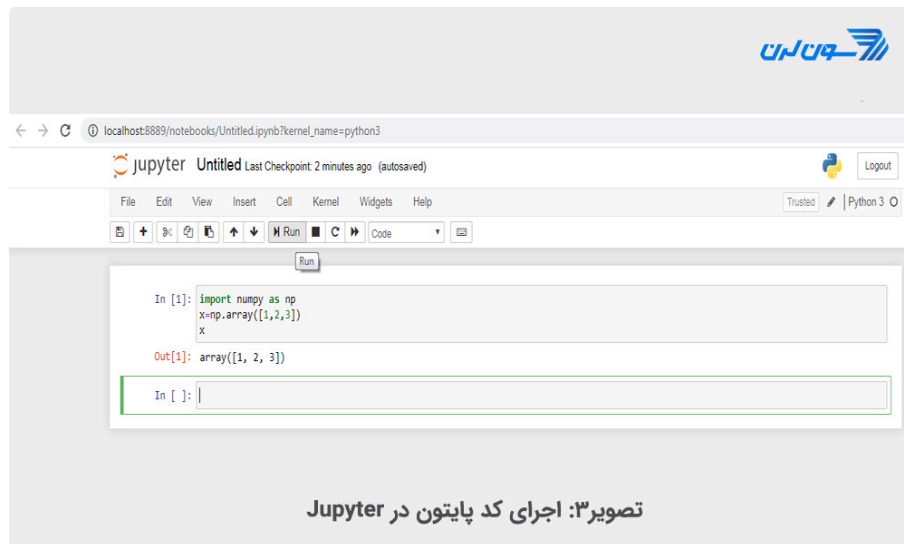
نصب این نرم افزار نکته ی خاصی ندارد و راحت است. تنها نکته ای که باید توجه کنید این است که شما پس از دانلود فایل و اجرای آن در طول نصب دو پنجره ی زیر را (پنجره ی سمت چپ را زودتر از سمت راست) مشاهده خواهید کرد. توجه کنید که در پنجره ی سمت چپ، تیک گزینه ی اول را طبق توضیحی که خود نرم افزار داده، علامت زنید. پنجره ی سمت راست مرحله ی نهایی نصب است، و دو گزینه ی آخر را تیک زنید.



پس از نصب نرم افزار (در ویندوز) در قسمت search از منوی start عبارت anaconda را تایپ کنید و Anaconda Navigator را انتخاب کنید. محیطی را برای شما باز می‌کند که انواع مختلفی از IDEها را در آن می‌بینید. Jupyter Notebook را انتخاب کنید. پس از آن وارد صفحه ای به شکل تصویر ۲ خواهید شد. از قسمت New گزینه ی Python را انتخاب کنید و وارد محیط کدنویسی شوید.



محیط کدنویسی به شکل تصویر ۳ است. پس از نوشتن کد خود، برای اجرا روی گزینه ی Run کلیک کنید.



اگر به دلیل حجم بالای آناکوندا (حدود ۵۰۰MB) نمی‌خواهید آن را نصب کنید می‌توانید از **miniconda** استفاده کنید که پکیجی خلاصه شده از برخی ویژگی‌های آناکوندا برای کار با پایتون است. پس از نصب آن برای نصب Jupyter Notebook دستور زیر را در محیط ترمینال یا خط فرمان وارد کنید:

```
conda install -c conda-forge notebook
```

اگر می‌خواهید از pip برای نصب استفاده کنید دستور زیر را بنویسید:

```
pip install notebook
```

برای اجرای ژوپیتتر نوتبوک از طریق خط فرمان می‌توانید دستور زیر را تایپ کنید:

```
jupyter notebook
```

آرایه در NumPy

آرایه‌ها در NumPy تاحدودی شبیه لیست در پایتون هستند اما در اصل با آن تفاوت دارند. اجازه بدهید به طور کامل این موضوع را تشریح کنیم. همان طور که از نام این کتابخانه مشخص است، NumPy از Numeric Python برگرفته شده و امکان کار با

اعداد را به طور گسترده فراهم می‌کند و آرایه ی NumPy ساختار داده ی این کتابخانه است. یعنی برای اینکه بتوانیم با داده‌ها در این کتابخانه کار کنیم باید آن‌ها را به شکل آرایه تبدیل کرد. در ادامه ی مقاله ی آموزش کامل کتابخانه NumPY ، با ایجاد یک آرایه ی numpy و عملیات ریاضی بر روی آرایه‌ها آشنا می‌شوید.

ایجاد یک آرایه ی NumPy

ساده‌ترین روش برای ساخت این آرایه استفاده از لیست پایتون است:

```
myPythonList = [1,9,8,3]
```

برای تبدیل لیست پایتون به آرایه ی NumPy از دستور np.array استفاده می‌کنیم.
(برای چاپ خروجی در ژوپیتر نام متغیر numpy_array_from_list را تایپ می‌کنیم.)

```
import numpy as np
numpy_array_from_list = np.array(myPythonList)
numpy_array_from_list
```

array([۱, ۹, ۸, ۳])

در عمل نیازی به تعریف لیست پایتونی به طور مجزا نیست و می‌توان به شکل زیر آرایه را تعریف کرد:

```
a = np.array([1,9,8,3])
```

توجه: در مستندات NumPy از np.darray هم برای ایجاد آرایه استفاده شده، اما روش ذکر شده مرسوم‌تر است. همچنین به جای لیست می‌توان از تاپل (tuple) نیز استفاده کرد.

عملیات ریاضی بر روی آرایه ها

عملیات ریاضی همچون جمع، تفریق، ضرب و تقسیم بر روی آرایه‌ها قابل انجام است. دستور کار به صورت استفاده از نام آرایه و عملگر موردنظر (*,+, -,) و عملوند است:

```
numpy_array_from_list + 10
```

```
array([۱۱, ۱۹, ۱۸, ۱۳])
```

توجه کنید که عدد ۱۰ با تمامی عناصر آرایه جمع زده شده است. در این خصوص مفهوم Broadcasting را در ادامه توضیح می‌دهیم.

Broadcasting یا انتشار همگانی در Numpy

واژه ی Broadcasting به چگونگی رفتار Numpy با آرایه هایی با Shape متفاوت در خلال عملگرهای محاسباتی اشاره دارد. به طور خلاصه، آرایه ی کوچکتر به اندازه ی آرایه ی بزرگتر پخش می‌شود تا به شکل و Shape یکسانی با آن تبدیل شود. اینکه از Broadcast استفاده بکنیم یا خیر؛ به شرایط داده و الگوریتم بستگی دارد و می‌تواند با توجه به آن‌ها به کارگیری این مفهوم در کارایی برنامه تأثیر مثبت یا منفی داشته باشد. Broadcasting در Numpy با توجه به محدودیت‌های مسأله و داده، کارایی را کنترل می‌کند.

قانون Broadcasting: باید اندازه ی محور نهایی یا آخر (یا همان مقدار آخرین Shape)، در هر دو آرایه داده یکسان باشد؛ یا مقدار بُعد آخر حداقل یکی از آرایه‌ها برابر یک باشد. حالت‌های زیر را در نظر بگیرید:

A(۲-D array): ۴ x ۳ # these numbers are the shape of array

B(۱-D array): ۳

Result : ۴ x ۳

A(۴-D array): ۷ x ۱ x ۶ x ۱

B(۳-D array): ۳ x ۱ x ۵

Result : ۷ x ۳ x ۶ x ۵

اما این یکی قابلیت Broadcasting ندارد. چون نه مقدار بُعد آخرشان یکی

است(یکی ۳ و دیگری ۰) و نه بُعد نهایی هیچ یک برابر عدد ۱ است:

A: ۴ x ۳

B: ۴

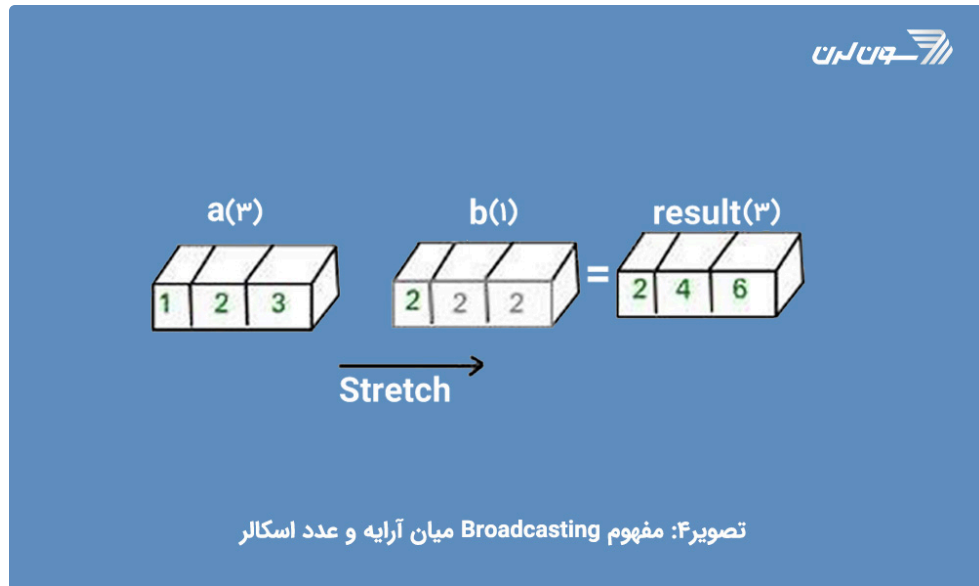
ساده‌ترین حالت Broadcast حین انجام عملیات محاسباتی میان یک آرایه و یک عدد اسکالر است.

```
import numpy as np
a = np.array([1.0, 2.0, 3.0])
# Example 1
b = 2.0
print(a * b)
# Example 2
c = [2.0, 2.0, 2.0]
print(a * c)
```

[۰.۶ ۰.۴ ۰.۲] [۰.۶ ۰.۴ ۰.۲]

می‌توان این گونه تصور کرد که عدد اسکالر به تعداد اندازه‌ی آرایه‌ای که قرار است با آن در محاسبات شرکت کند، کپی می‌شود. البته این یک توصیف مفهومی از کار است (تصویر ۴) و Numpy با توجه به هوشمندی که در طراحی‌اش به کار رفته، برای حفظ

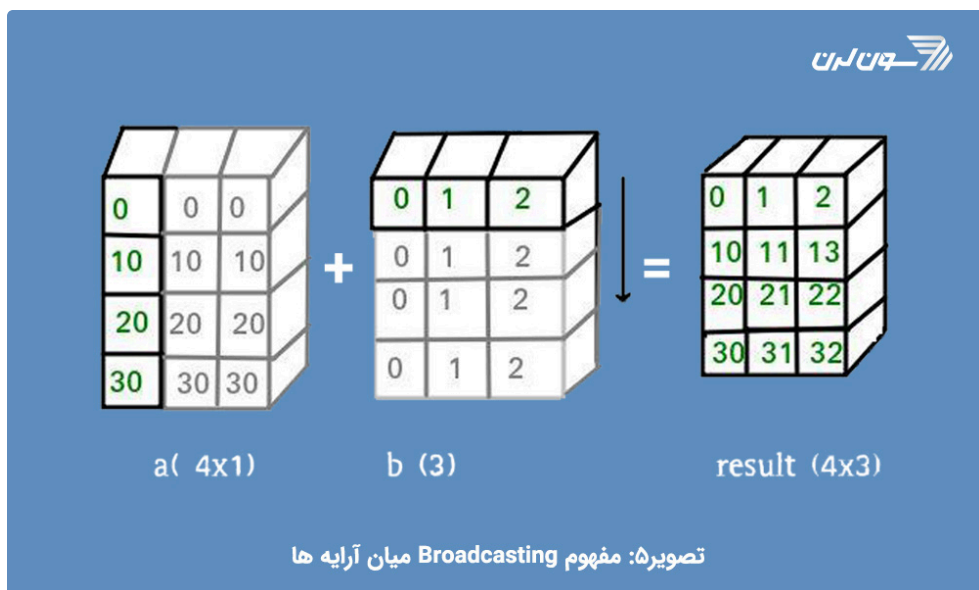
کارایی عملکرد و حافظه، چنین کپی ای از آن عدد اسکالر ایجاد نمی‌کند.



حال مثالی را بررسی می‌کنیم که هر دو آرایه باید گسترش یابند:

```
import numpy as np
a = np.array([0.0, 10.0, 20.0, 30.0])
b = np.array([0.0, 1.0, 2.0])
print(a[:, np.newaxis] + b)
```

`[[0. 1. 2.] [10. 11. 13.] [20. 21. 22.] [30. 31. 32.]]`



شکل آرایه (Shape of Array)

در این بخش از مقاله ی آموزش کامل کتابخانه NumPY ، شکل آرایه را توضیح خواهیم داد، که می‌تواند یک بعدی، دوبعدی، یا سه بعدی باشد. شکل آرایه را می‌توان با دستور shape. بدست آورد و نیز با دستور dtype. می‌توان نوع داده‌های آرایه و مقدار فضای اختصاص یافته به آن را فهمید.

```
import numpy as np
a = np.array([1,2,3])
print(a.shape)
print(a.dtype)
#### Different type
b = np.array([1.1,2.0,3.2])
```

(۳,) int64

float64

آرایه ی دو بعدی

بعد دوم را می‌توان با افزودن یک کاما "," تعریف کرد. البته توجه کنید که داده‌های پس از کاما هم داخل براکت باز و بسته [] (ویا پرانتز) اگر که تاپل باشد) باید قرار گیرند.

```
### 2 dimension
c = np.array([(1,2,3),
              (4,5,6)])
print(c.shape)
```

(۲, ۳)

آرایه ی سه بعدی

آرایه‌ها با ابعاد بالاتر به شکل زیر تعریف می‌شوند: (دوباره یک کاما و یک براکت باز و بسته ی دیگر افزوده می‌شود).

```
### 3 dimension
d = np.array([
    [[1, 2, 3],
     [4, 5, 6]],
    [[7, 8, 9],
     [10, 11, 12]]
])
print(d.shape)
```

(۳, ۲, ۲)

اگر بخواهید فضای اشغال شده توسط آرایه را به دست آورید از تابع `itemsize` استفاده کنید. به مثال زیر توجه کنید:

```
x = np.array([1, 2, 3], dtype=np.complex128)
x.itemsize
```

element x has ۱۶ bytes # ۱۶

آرایه با مقادیر صفر و یا با مقادیر یک در NumPy

می‌توان با دستورات `np.zeros` و `np.ones` به ترتیب ماتریس‌هایی ایجاد کرد که تمامی عناصر آن صفر و یک باشند. چنین ماتریس‌هایی می‌توانند به طور مثال برای مقداردهی اولیه ی ماتریس وزن‌ها در شبکه ی عصبی مورد استفاده قرار گیرند. قاعده ی کلی به شکل زیر است:

```
numpy.zeros(shape, dtype=float, order='C')
numpy.ones(shape, dtype=float, order='C')
```

shape شکل آرایه را تعیین می‌کند و dtype یا همان datatype نوع داده را مشخص می‌کند و پارامتری اختیاری است که به طور پیش فرض مقدار آن float64 است. پارامتر order هم اختیاری است و مقدار پیش فرض C را می‌گیرد و یک استایل پایه برای سطرهای ماتریس است. در زیر چند مثال را می‌بینیم.

```
#Example numpy zero
import numpy as np
np.zeros((2,2))

#Example numpy zero with datatype
np.zeros((2,2), dtype=np.int16)

#Example numpy one 2D Array with datatype
np.ones((1,2,3), dtype=np.int16)
```

خروجی‌ها به ترتیب زیر هستند:

```
array([[0., 0.], [0., 0.]]) array([[0, 0], [0, 0]], dtype=int16) array([[[[1, 1, 1], [1, 1, 1]]],
dtype=int16)
```

numpy.reshape و numpy.flatten در پایتون

گاهی شما نیاز به تغییر (reshape) داده‌ها از فرم عرضی به طولی دارید. برای این کار از تابع reshape به شکل زیر استفاده می‌کنیم: (a ورودی ماست).

```
numpy.reshape(a, newShape, order='C')
```

```
import numpy as np
e = np.array([(1,2,3), (4,5,6)])
print(e)
e.reshape(3,2)
```

Before reshape `[[1 2 3] [4 5 6]]` //After Reshape `array([[1, 2], [3, 4], [5, 6]])`

گاهی در حین کار با شبکه‌های عصبی نیاز دارید تا آرایه ی خود را مسطح (flat) کنید. برای این کار از `flatten` به شکل زیر استفاده می‌کنیم:

```
numpy.flatten(order='C')
```

```
e.flatten()
```

```
array([۱, ۲, ۳, ۴, ۵, ۶])
```

اتصال آرایه‌ها به هم در NumPy (Stacking)

چندین آرایه می‌توانند در امتداد محورهای مختلف به همدیگر متصل شوند. که در این بخش از مقاله ی آموزش کامل کتابخانه NumPY ، انواع اتصال آرایه‌ها به هم را توضیح می‌دهیم.

`np.vstack`: آرایه‌ها را به طور عمودی به هم وصل می‌کند.

`np.hstack`: آرایه‌ها را به طور افقی متصل می‌کند.

`np.column_stack`: برای اتصال آرایه ی یک بُعدی به عنوان ستون به سطر آرایه ی دوبعدی.

`np.concatenate`: برای اتصال آرایه‌ها به همدیگر در خلال یک محور معین (یعنی

محور یا همان `axis` به عنوان پارامتر ورودی تابع تعریف می‌شود).

مثال زیر نحوه ی کارکرد هر یک را نشان می‌دهد.

```

import numpy as np
a = np.array([[1, 2],
               [3, 4]])
b = np.array([[5, 6],
               [7, 8]])

# vertical stacking
print("Vertical stacking:\n", np.vstack((a, b)))

# horizontal stacking
print("\nHorizontal stacking:\n", np.hstack((a, b)))

c = [5, 6]

# stacking columns
print("\nColumn stacking:\n", np.column_stack((a, c)))

# concatenation method
print("\nConcatenating to 2nd axis:\n", np.concatenate((a, b

```

Vertical stacking: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$ Horizontal stacking: $\begin{bmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \end{bmatrix}$ Column stacking: $\begin{bmatrix} 1 & 2 & 5 \\ 3 & 4 & 6 \end{bmatrix}$ Concatenating to 2nd axis: $\begin{bmatrix} 1 & 2 & 5 & 6 \\ 3 & 4 & 7 & 8 \end{bmatrix}$

تقسیم آرایه در NumPy (Splitting)

در این بخش از مقاله ی آموزش کامل کتابخانه NumPY ، توابعی برای تقسیم آرایه‌ها را معرفی می‌کنیم.

توابعی برای تقسیم آرایه‌ها تعریف شده که به شرح زیر است:

`np.hsplit`: تقسیم آرایه به شکل افقی.

`np.vsplit`: تقسیم آرایه به شکل عمودی.

`np.array_split`: آرایه را در خلال امتداد تعیین شده در ورودی (با پارامتر `axis`)

تقسیم می‌کند.

```
import numpy as np
a = np.array([[1, 3, 5, 7, 9, 11],
              [2, 4, 6, 8, 10, 12]])
# horizontal splitting
print("Splitting along horizontal axis into 2 parts:\n", np.
# vertical splitting
print("\nSplitting along vertical axis into 2 parts:\n", np.
```

Splitting along horizontal axis into ۲ parts: [array([[۱, ۳, ۵], [۲, ۴, ۶]]),
array([[۷, ۹, ۱۱], [۸, ۱۰, ۱۲]])] Splitting along vertical axis into ۲ parts:
[array([[۱, ۳, ۵, ۷, ۹, ۱۱]]), array([[۲, ۴, ۶, ۸, ۱۰, ۱۲]])]

پیمایش آرایه در NumPy

پیمایش آرایه یا Iterate کردن آن یعنی عناصر آن را یکی یکی مشاهده کنیم. چون با آرایه‌های چند بعدی در numpy سروکار داریم، پیمایش را می‌توان با حلقه ی for در پایتون انجام داد. آرایه ی یک بعدی (۱D) را به شکل زیر پیمایش می‌کنیم:

```
import numpy as np
arr = np.array([1, 2, 3])
for x in arr:
    print(x)
```

۳ ۲ ۱

در آرایه ی دو بعدی (۲D) پیمایش به صورت سطر به سطر انجام می‌شود:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    print(x)
```

[۳ ۲ ۱] [۶ ۵ ۴]

اگر بخواهیم مقادیر به صورت اسکارل برگردانده شود، به شکل زیر عمل می‌کنیم.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    for y in x:
        print(y)
```

۶ ۵ ۴ ۳ ۲ ۱

در پیمایش آرایه ی سه بعدی (۳D) آرایه‌های دوبعدی آن را جدا می‌کنیم:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
    print(x)
```

[[۳ ۲ ۱] [۶ ۵ ۴]] [[۹ ۸ ۷]] [[۱۲ ۱۱ ۱۰]]

برای برگرداندن هر عنصر نیز باید آرایه را در هر بعد، جداگانه پیمایش کنیم:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
    for y in x:
        for z in y:
            print(z)
```

۱۲ ۱۱ ۱۰ ۹ ۸ ۷ ۶ ۵ ۴ ۳ ۲ ۱

پیمایش آرایه با **nditer**

تابع `nditer` تابعی است که می‌تواند از ساده‌ترین تا پیچیده‌ترین حالت پیمایش را اجرا کند و بسیاری از مشکلات ما را در زمینه `Iteration` حل می‌کند. به طور مثال برای استخراج عناصر آرایه `y` سه بعدی به شکل اسکالر می‌توان با این تابع به شکل زیر عمل کرد و درگیر پیچیدگی `for` های تودرتو نشد:

```
import numpy as np
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
for x in np.nditer(arr):
    print(x)
```

۸ ۷ ۶ ۵ ۴ ۳ ۲ ۱

پیمایش آرایه با نوع داده ای مختلف

می‌توان از آرگومان `op_dtypes` استفاده کرد و نوع داده‌ها را به نوع داده `y` مورد نظر حین پیمایش تغییر داد. `Numpy` قادر نیست به طور درجا نوع داده‌ها را تغییر دهد و برای این کار به فضای اضافی نیاز دارد که این فضای اضافه بافر نام دارد و برای فعال سازی بافر در تابع `nditer` از `flag='buffered'` استفاده می‌کنیم.

```
import numpy as np
arr = np.array([1, 2, 3])
for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):
    print(x)
```

'b'۱' b'۲' b'۳'

پیمایش با گام‌های متفاوت

می‌توانیم داده‌ها را به‌طور فیلتر شده پیمایش کنیم. به‌طور مثال در آرایه‌ی دوبعدی زیر یک در میان عناصر را خواند.

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for x in np.nditer(arr[:, ::2]):
    print(x)
```

۷ ۵ ۳ ۱

پیمایش با اندیس‌ها

گاهی‌حین پیمایش عناصر به اندیس عنصر مورد نظر نیاز داریم. متد `ndenumerate` برای این منظور به کار می‌رود.

```
import numpy as np
arr = np.array([1, 2, 3])
for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

۳ (۰,۲) ۲ (۰,۱) ۱ (۰,۰)

مثالی از آرایه‌ی دو بعدی را هم در این خصوص ببینید:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

۸ (۳,۱) ۷ (۲,۱) ۶ (۱,۱) ۵ (۰,۱) ۴ (۳,۰) ۳ (۲,۰) ۲ (۱,۰) ۱ (۰,۰)

جبرخطی در NumPy

در ادامه ی مقاله ی آموزش کامل کتابخانه NumPY به آموزش جبر خطی در NumPY می‌پردازیم، ماژول جبرخطی در NumPy متدهای بسیاری را برای اعمال روی انواع آرایه‌ها در NumPy فراهم می‌کند. شما با این ماژول قادر هستید موارد زیر را محاسبه کنید:

مرتبه، اثر ماتریس (trace of matrix) و دترمینان یک آرایه
مقادیر ویژه ی یک ماتریس (eigen values)
ضرب‌های ماتریسی و برداری (ضرب نقطه ای، داخلی، خارجی و ...)
حل معادلات خطی یا تانسور
مثال زیر چگونگی اعمال عملیات جبری را نشان می‌دهد.

```
import numpy as np
A = np.array([[6, 1, 1],
              [4, -2, 5],
              [2, 8, 7]])
print("Rank of A:", np.linalg.matrix_rank(A))
print("\nTrace of A:", np.trace(A))
print("\nDeterminant of A:", np.linalg.det(A))
print("\nInverse of A:\n", np.linalg.inv(A))
print("\nMatrix A raised to power 3:\n", np.linalg.matrix_po
```

Rank of A: ۳ Trace of A: ۱۱ Determinant of A: -۳۰۶.۰ Inverse of A: [[
۰.۱۷۶۴۷۰۵۹ -۰.۰۰۳۲۶۷۹۷ -۰.۰۲۲۸۷۵۸۲] [۰.۰۵۸۸۲۳۵۳ -۰.۱۳۰۷۱۸۹۵
۰.۰۸۴۹۶۷۳۲] [-۰.۱۱۷۶۴۷۰۶ ۰.۱۵۰۳۲۶۸ ۰.۰۵۲۲۸۷۵۸]] Matrix A raised to
power ۳: [[۳۳۶ ۱۶۲ ۲۲۸] [۴۰۶ ۱۶۲ ۴۶۹] [۶۹۸ ۷۰۲ ۹۰۵]]

تصور کنید می‌خواهیم این معادله ی خطی را محاسبه کنیم.

$$x + ۲*y = ۸$$

$$x + ۴*y = ۱۸*۳$$

این معادله با به کارگیری متد `linalg.solve` قابل حل است.

```
import numpy as np
# coefficients
a = np.array([[1, 2], [3, 4]])
# constants
b = np.array([8, 18])
print("Solution of linear equations:", np.linalg.solve(a, b))
```

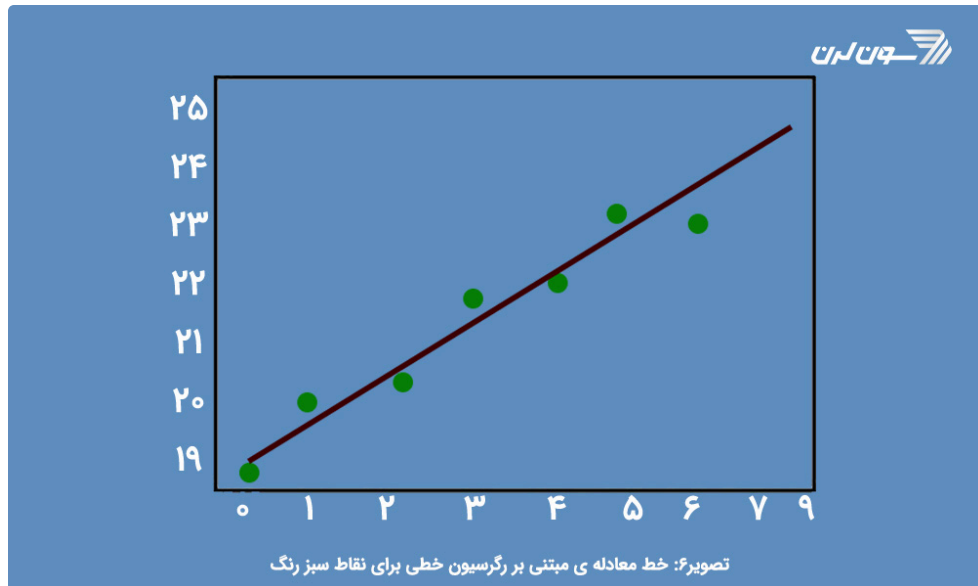
Solution of linear equations: [۲. ۳.]

در آخر مثالی را بررسی می‌کنیم که چگونگی محاسبه ی رگرسیون خطی را با استفاده از روش کاهش مربعات خطا نشان می‌دهد. یادآوری

رگرسیون خطی، خطی است که به فرم $w_1x_i + w_2 = y_i$ است و سعی دارد w_i ها را طوری تعیین کند که مجموع مربعات فاصله ی نقاطی مانند y_i از خط حاصل به کمترین مقدار برسد. (y_i مقدار حقیقی برای صفات x_i است و $w_1x_i + w_2$ مقدار تخمینی توسط رگرسیون خطی است.)

```
import numpy as np
import matplotlib.pyplot as plt
# x co-ordinates
x = np.arange(0, 9)
A = np.array([x, np.ones(9)])
# linearly generated sequence
y = [19, 20, 20.5, 21.5, 22, 23, 23, 25.5, 24]
# obtaining the parameters of regression line
w = np.linalg.lstsq(A.T, y)[0]
# plotting the line
line = w[0]*x + w[1] # regression line
plt.plot(x, line, 'r-')
plt.plot(x, y, 'o')
plt.show()
```

خروجی این معادله به شکل زیر است:



تولید اعداد تصادفی در NumPy

برای تولید اعداد تصادفی در NumPy به طور کلی از تابع random استفاده می‌کنیم و می‌توانیم برای اینکه این اعداد تصادفی در یک توزیع خاص آماری با ویژگی‌های مورد نظر ما باشند، از توابع دیگری نیز کمک بگیریم.

`numpy.random.rand(d0,d1,...,dn)`: آرایه ای در ابعاد مشخص ایجاد می‌کند و

درایه‌های آن را با اعداد تصادفی پر می‌کند. `d0,d1,...,dn` ابعاد آرایه را مشخص

می‌کنند و اعدادی صحیح و اختیاری اند. اگر آرگومانی داده نشود یک مقدار اعشاری تولید می‌کند.

`numpy.random.normal(loc, scale, size)`: آرایه ای در شکل و ابعاد مشخص

تولید می‌کند و درایه‌های آن را با مقادیر تصادفی که بخشی از یک توزیع نرمال

(گوسی) محسوب می‌شوند، پر می‌کند. به این توزیع به خاطر شکلی که دارد، منحنی

زنگ هم می‌گویند. و اما پارامترهای این تابع:

`loc`: میانگین توزیع.

`scale`: انحراف معیار توزیع.

`size`: تعداد نمونه داده هاست.

مثال‌های زیر را در نظر بگیرید:

```
# numpy.random.rand() method
import numpy as np
# 1D Array
array = np.random.rand(5)
print("1D Array filled with random values : \n", array)
```

1D Array filled with random values : [۰.۸۴۵۰۳۹۶۸ ۰.۶۱۵۷۰۹۹۴

۰.۷۶۱۹۹۴۵ ۰.۳۴۹۹۴۸۰۳ ۰.۴۰۱۱۳۷۶۱]

در مثال بعدی ای یک بُعدی که از توزیع گوسی پیروی می‌کند، تولید می‌کنیم:

```
import numpy as np
# 1D Array
array = np.random.normal(0.0, 1.0, 5)
print("1D Array filled with random values "
      "as per gaussian distribution : \n", array)
# 3D array
array = np.random.normal(0.0, 1.0, (2, 1, 2))
print("\n\n3D Array filled with random values "
      "as per gaussian distribution : \n", array)
```

1D Array filled with random values as per gaussian distribution :

[-۰.۹۹۰۱۳۱۷۲ -۱.۵۲۵۲۱۸۰۸ ۰.۳۷۹۵۵۶۸۴ ۰.۵۷۸۵۹۲۸۳ ۱.۳۴۳۳۶۸۶۳] ۳D

Array filled with random values as per gaussian distribution :

[[[-۰.۰۳۲۰۳۷۴ ۲.۱۴۹۷۷۸۴۹]] [[۰.۳۷۸۹۵۸۵ ۰.۱۷۶۹۲۱۲۵]]]

مثال زیر نیز تفاوت تولید اعداد تصادفی در حالت معمول با حالتی که از توزیع نرمال

پیروی می‌کند را نمایش می‌دهد (از کتابخانه ی matplotlib برای رسم نمودار کمک

گرفته ایم):

```
import numpy as np
import matplotlib.pyplot as plot

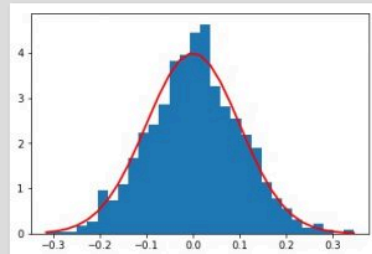
# 1D Array as per Gaussian Distribution
mean = 0
std = 0.1
array = np.random.normal(0, 0.1, 1000)
print("1D Array filled with random values "
      "as per gaussian distribution : \n", array);

# Source Code :
# https://docs.scipy.org/doc/numpy-1.13.0/reference/
# generated/numpy-random-normal-1.py
count, bins, ignored = plot.hist(array, 30, normed=True)
plot.plot(bins, 1/(std * np.sqrt(2 * np.pi)) *
          np.exp( - (bins - mean)**2 / (2 * std**2) ),
          linewidth=2, color='r')
plot.show()

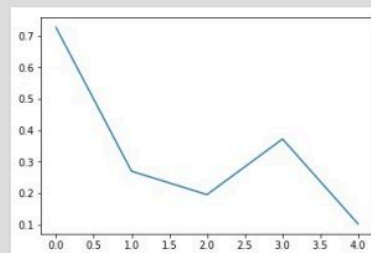
# 1D Array constructed Randomly
random_array = np.random.rand(5)
print("1D Array filled with random values : \n", random_array)
plot.plot(random_array)
plot.show()
```

خروجی این کد به شکل زیر است:

```
1D Array filled with random values as per gaussian distribution :  
[ 0.12413355  0.01868444  0.08841698 ..., -0.01523021 -0.14621625  
-0.09157214]
```



```
1D Array filled with random values :  
[ 0.72654409  0.26955422  0.19500427  0.37178803  0.10196284]
```



توابع دیگری نیز برای تولید اعداد تصادفی با ویژگی‌های مختلف وجود دارد که پیشنهاد می‌دهیم آن‌ها را از [دایکومنت مربوطه](#)، مطالعه کنید.

کار با تاریخ و زمان در NumPy

در این بخش از مقاله ی آموزش کامل کتابخانه NumPY کار با تاریخ و زمان در NumPY را آموزش می‌دهیم، NumPy دارای انواعی از ساختارهای داده (Data Structure) است که به طور طبیعی از عملکرد datetime پشتیبانی می‌کنند. این نوع داده 'datetime64' نامیده می‌شود که از کتابخانه ای در زبان پایتون با همین نام برگرفته شده است. در مثال زیر چند نمونه از کاربرد زمان و تاریخ را می‌بینید.

```

import numpy as np

# creating a date
today = np.datetime64('2017-02-12')
print("Date is:", today)
print("Year is:", np.datetime64(today, 'Y'))

# creating array of dates in a month
dates = np.arange('2017-02', '2017-03', dtype='datetime64[D]')
print("\nDates of February, 2017:\n", dates)
print("Today is February:", today in dates)

# arithmetic operation on dates
dur = np.datetime64('2017-05-22') - np.datetime64('2016-05-22')
print("\nNo. of days:", dur)
print("No. of weeks:", np.timedelta64(dur, 'W'))

# sorting dates
a = np.array(['2017-02-12', '2016-10-13', '2019-05-22'], dtype='datetime64[D]')
print("\nDates in sorted order:", np.sort(a))

```

```

Date is: 2017-02-12 Year is: 2017 Dates of February, 2017: [

```

numpy.asarray در پایتون

تابع `as array` برای تبدیل داده ی ورودی به آرایه استفاده می شود. ورودی می تواند به شکل لیست، تاپل و یا `ndarray` و... باشد. فرم کلی به شکل زیر است:

```

numpy.asarray(data, dtype=None, order=None)

```

`data`: داده ای که می خواهید به آرایه تبدیل کنید.

`dtype`: یک پارامتر اختیاری است و اگر مقداردهی نشود، مقدار آن از نوع داده های ورودی گرفته می شود.

order: یک پارامتر اختیاری است و مقدار پیش فرض آن C است و استایل پایه برای سطرهاست و مقدار دیگری که می‌تواند بگیرد F است. ماتریس دو بُعدی زیر را که شامل چهار سطر و ستون پرشده با یک است در نظر بگیرید:

```
import numpy as np
A = np.matrix(np.ones((4,4)))
```

نمی‌توان مقدار درایه‌های این ماتریس را با کپی و انتساب به شکل زیر تغییر داد:

```
np.array(A)[2]=2
print(A)
```

```
[[1. 1. 1. 1.] [1. 1. 1. 1.] [1. 1. 1. 1.] [1. 1. 1. 1.]
```

ماتریس تغییرناپذیر است. اگر بخواهید تغییری در مقادیر درایه‌ها داشته باشید، می‌توانید از تابع `asarray` استفاده کنید و مثلاً مقادیر سطر سوم را به دو تغییر دهیم:

```
np.asarray(A)[2]=2
print(A)
```

```
[new value [1. 1. 1. 1.] # [2. 2. 2. 2.] [1. 1. 1. 1.] [1. 1. 1. 1.]
```

numpy.arange در پایتون

این تابع در NumPy آرایه ای را برمی گرداند که نتیجه ی آن بازه ای از اعداد است که فاصله ی بین آن‌ها برابر است. قاعده ی کلی دستور به شکل زیر است:

`numpy.arange(start, stop, step)`

start: شروع بازه.

stop: پایان بازه (که خود این عدد را شامل نمی‌شود).

step: فاصله ی میان اعداد در بازه که به طور پیش فرض یک است.

array([۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸, ۹, ۱۰])

مثال دیگر:

```
import numpy np
np.arange(1, 14, 4)
```

array([۱, ۵, ۹, ۱۳])

numpy.linspace و numpy.logspace در پایتون

این تابع نیز دنباله ای از اعداد را تولید می کند و قاعده ی تعریف آن به صورت زیر

است: `numpy.linspace(start, stop, num, endpoint)`

`start`: مقدار شروع دنباله.

`stop`: مقدار پایان دنباله (خود مقدار پایانی در نظر گرفته می شود).

`num`: تعداد اعداد مورد نظر که مقدار پیش فرض ۵۰ را دارد.

`endpoint`: اگر `True` باشد، (که به طور پیش فرض هست) مقدار `stop` را به عنوان

مقدار نهایی در نظر می گیرد و اگر `False` بود آن را در نظر نمی گیرد.

به عنوان مثال می توان ده مقدار با فاصله ی یکسان در بازه ی ۱ تا ۵ به شکل زیر

تولید کرد:

```
import numpy as np
np.linspace(1.0, 5.0, num=10)
```

array([1. , 1.44444444, 1.88888889, 2.33333333, 2.77777778,
3.22222222, 3.66666667, 4.11111111, 4.55555556, 5.])

اگر نمی خواهید آخرین عدد در بازه جزو اعداد تولیدی باشد، همان طور که گفته شد

مقدار `endpoint` را برابر `False` قرار دهید:

```
np.linspace(1.0, 5.0, num=5, endpoint=False)
```

```
array([۱. , ۱.۸, ۲.۶, ۳.۴, ۴.۲])
```

np.logspace هم مشابه linspace عمل می‌کند با این تفاوت که توان اعداد را محاسبه می‌کند و پایه به طور پیش فرض برابر عدد ۱۰ است. قاعده ی آن به شکل زیر است: numpy.logspace(start, stop, num, endpoint):
زیر بررسی می‌کنیم:

```
np.logspace(3.0, 4.0, num=4)
```

```
array([ ۱۰۰۰. , ۲۱۵۴.۴۳۴۶۹۰۰۳, ۴۶۴۱.۵۸۸۸۳۳۶۱, ۱۰۰۰۰. ])
```

```
np.logspace(4.0, 5.0, num=3, endpoint=False)
```

```
array([ ۱۰۰۰۰. , ۲۱۵۴۴.۳۴۶۹۰۰۳۲, ۴۶۴۱۵.۸۸۸۳۳۶۱۳])
```

```
np.logspace(4.0, 5.0, num=3, base=2.0)
```

```
array([ ۱۶. , ۲۲.۶۲۷۴۱۷, ۳۲. ])
```

اندیس دهی (Indexing) و برش (Slicing) آرایه‌های NumPy در پایتون

برش ماتریس و یا آرایه توسط NumPy کار ساده ای است. به مثال‌های زیر توجه کنید:

```
## Slice
import numpy as np
e = np.array([(1,2,3), (4,5,6)])
print(e)
```

```
[[۶ ۵ ۴],[۳ ۲ ۱]]
```

```
print('First row:', e[0])  
print('Second row:', e[1])
```

First row: [۱ ۲ ۳] Second row: [۴ ۵ ۶]

یادآوری: در زبان پایتون داریم:

مقدار قبل از کاما در براکت، به سطر اشاره می‌کند.

مقدار سمت راست کاما به ستون اشاره می‌کند.

اگر بخواهید تنها ستون را انتخاب کنید، قبل از شماره ی ستون باید دو نقطه (:) قرار دهید [column_index:] .

دو نقطه [column_index:] به این معنی است که شما تمامی سطرهای آن ستون را انتخاب کرده اید.

```
print('Second column:', e[:,1])
```

Second column: [۲ ۵]

برای برگرداندن دو مقدار اول از سطر دوم (اندیس یکم) آرایه ی e دستور را به شکل

زیر می‌نویسند: (توجه کنید باید دو نقطه را قرار دهید تا همه ی مقادیر پیش از

ستون سوم یعنی اندیس دوم، با اندیس‌های صفر و یک، انتخاب شوند).

```
## Second Row, two values  
print(e[1, :2])  
#result  
[4 5]
```

توابع آماری NumPy

این کتابخانه تعدادی تابع آماری برای محاسبه ی مواردی همچون مینیمم، ماکزیمم،

میانگین، میانه، دهک و صدک، انحراف معیار، واریانس و ... برای اعمال روی آرایه‌ها

و ماتریس‌های ورودی دارد. در جدول زیر برخی از این توابع آمده اند:



تابع	NumPy
Min	np.min()
Max	np.max()
Mean	np.mean()
Median	np.median()
Standard deviation	np.std()

جدول ۱: توابع محاسبات آماری در NumPy

آرایه ی زیر را در نظر بگیرید:

```
import numpy as np
normal_array = np.random.normal(5, 0.5, 10)
print(normal_array)
```

```
۵.۶۱۲۱۱۳۱۷ ۴.۸۵۱۶۵۵۶۷ ۴.۹۴۶۶۵۹ ۴.۶۵۳۹۲۷۶۷ ۴.۸۴۲۳۳۵۵۸ ۵.۵۶۱۷۱۸۵۲]
[۴.۶۸۷۳۱۱۲۵ ۴.۴۹۸۸۸۹۳۶ ۵.۲۲۶۷۵۷۳۶ ۴.۴۶۷۰۴۲۴۴
```

توابع آماری را روی داده‌های تولید شده به شکل زیر اعمال می‌کنیم:

```
### Min
print(np.min(normal_array))
### Max
print(np.max(normal_array))
### Mean
print(np.mean(normal_array))
### Median
print(np.median(normal_array))
### Std
print(np.std(normal_array))
```

ضرب در پایتون به کمک NumPy

numpy.dot ضرب نقطه ای بین ماتریس‌ها یا همان ضرب ماتریسی را انجام می‌دهد.

قاعده ی کلی به شکل زیر است:

```
numpy.dot(x, y, out=None)
```

x و y آرایه ی ورودی اند که باید ۱D یا ۲D (یک بُعدی یا دو بُعدی) باشند. خروجی اگر

ورودی آرایه یک بُعدی باشد، مقدار اسکالر و در غیر این صورت یک آرایه است. به

مثال زیر توجه کنید:

```
## Linear algebra
### Dot product: product of two arrays
f = np.array([1,2])
g = np.array([4,5])
### 1*4+2*5
np.dot(f, g)
```

۱۴

اگر ورودی‌ها به صورت اسکالر باشند و یا ضرب عضو به عضو اعضای آرایه‌ها مدنظر

باشد از numpy.multiply می‌توان استفاده کرد. اما اگر ضرب ماتریسی برای آرایه‌های

دوبعدی می‌خواهیم انجام دهیم، numpy.matmul ترجیح داده می‌شود.

تفاوت میان View و Copy در NumPy

تفاوت میان یک کپی و یک view از یک آرایه در این است که کپی یک آرایه ی جدید

است اما؛ view فقط یک دید از آرایه ایجاد می‌کند. کپی مسئول داده‌های خود است

و هر گونه تغییر داده در آن، مستقل از آرایه ی اولیه است و بالعکس، هر تغییری در آرایه ی اولیه در کپی آن بی تأثیر است. درمورد view، این قضیه صادق نیست و تغییرات داده‌ها در view و آرایه ی اولیه بر یکدیگر اثر می‌گذارد.

Copy

یک کپی از آرایه ایجاد می‌کنیم و تغییرات آرایه و کپی آن را بر همدیگر مشاهده می‌کنیم:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42
print(arr)
print(x)
```

[۵ ۴ ۳ ۲ ۱] [۵ ۴ ۳ ۲ ۴۲]

View

یک view از آرایه ایجاد می‌کنیم و تغییرات آرایه و view آن را بر همدیگر مشاهده می‌کنیم:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
x[0] = 31
print(arr)
print(x)
```

[۵ ۴ ۳ ۲ ۳۱] [۵ ۴ ۳ ۲ ۳۱]

همان طور که گفته شد کپی بر داده‌های خود مالکیت دارد اما view نه. برای بررسی مالکیت آرایه ای در NumPy ، می‌توان مقدار ویژگی base آن را بررسی کرد. اگر None بود، یعنی مالکیت دارد و اگر به آبجکت (Object) اصلی و اولیه اشاره کرد یعنی مالکیت ندارد.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
y = arr.view()
print(x.base)
print(y.base)
```

None [۱ ۲ ۳ ۴ ۵]

نتیجه گیری

در مقاله ی آموزش کامل **کتابخانه NumPY** ، با توجه به اهمیت انجام محاسبات در علم داده با توابع کتابخانه ی NumPy که از پکیج‌های اساسی در ساخت سایر پکیج‌های داده کاوی با پایتون است، آشنا شدیم. NumPy به عنوان کتابخانه ای با استفاده ی گسترده در محاسبات نقشی مهم در سایر کتابخانه‌های محاسباتی پایتون همچون opencv، matplotlib، tensorflow، sikit-learn و... دارد. داشتن درک درستی از این کتابخانه در فهم سایر کتابخانه‌های پیشرفته‌تر بسیار مفید خواهد بود. بی صبرانه منتظر نظرات شما خوانندگان عزیز هستیم و امیدواریم همراه ما در ادامه ی مباحث مقالات آموزشی علم داده باشید.

هدف مجموعه سون لرن افزایش سطح کیفیت آموزش و ساختن راهی برای
ورود دانشجویان به بازار کار تخصصی است
:: زیرساخت سون لرن تامین شده توسط همراه خوبمون پارس پک.

دوره‌های آموزشی درباره ما قوانین و مقررات



تهران - منطقه ۵ - بلوار اباذر - پیامبر شرقی - مجتمع گلشن - واحد ۵۰۶

